

Dynamic Reconfiguration Guide (http)

Table of Contents

1. Overview	2
1.1. nginx.http at a glance	3
1.2. Lifecycle phases	3
2. nginx.createSocket(address)	4
2.1. Signature	4
2.2. Behaviour	4
2.3. NginxSocket properties and methods	4
2.4. Example: create a socket in config phase	5
3. sock.broadcast()	5
3.1. Signature	5
3.2. Why broadcast is needed	5
3.3. Full broadcast protocol	6
3.4. nginx.onSocket — the receiver callback	7
3.5. Timing and sequencing	8
3.6. The bcast channel is activated lazily	8
3.7. Errors and constraints	9
3.8. Example: minimal post-fork port provisioning	9
3.9. Example: broadcast with confirmation via SharedWorker	9
3.10. Example: broadcast + suspend for atomic rollout	11
4. nginx.http.attach(sock)	12
4.1. Signature	12
4.2. Behaviour	12
4.3. NginxHttpListener properties and methods	12
4.4. Full Phase A-D example	12
4.4.1. Flow diagram	13
4.5. Post-fork socket creation (Phase F)	13
5. nginx.http.servers[]	14
6. nginx.http.sockets[]	14
7. nginx.http.variables{}	14
8. Hash size parameters	15
9. nginx.http.addServer(name[, opts])	16
9.1. Signature	16
9.2. Two-phase commit	16
10. nginx.http.removeServer(name)	16

10.1. Signature	17
11. <code>nginx.http.rebuildVhostDispatch()</code>	17
11.1. Signature	17
12. <code>nginx.http.match(uri[, serverName])</code>	17
12.1. Signature	17
13. Complete examples	18
13.1. Example 1: Dynamic HTTP endpoint provisioning	18
13.2. Example 2: Inspect all sockets and servers at startup	19
13.3. Example 3: Provision a new port at config time (Phases A-D)	20
13.4. Example 4: Variable introspection in a request handler	21
13.5. Example 5: Cross-worker server provisioning via <code>SharedWorker</code>	21
13.6. Example 6: <code>match()</code> as a config validation tool	22
14. Cross-worker propagation	23
14.1. Atomic multi-server reconfiguration across all workers	23
15. API Reference	24
15.1. <code>nginx</code> (global)	24
15.2. <code>NginxHttp</code> (<code>nginx.http</code>)	24
15.3. <code>NginxSocket</code>	25
15.4. <code>nginx.onSocket</code> (callback)	25
15.5. <code>NginxHttpListener</code>	25
16. Design notes	26
16.1. Why four phases?	26
16.2. Two-phase commit for server names	26
16.3. <code>variables{}</code> is read-only	26
16.4. Post-fork socket creation (Phase F)	26
16.5. <code>broadcast()</code> is synchronous, <code>nginx.onSocket</code> is asynchronous	26
16.6. The bcast channel is a pre-fork <code>socketpair</code>	27



DRAFT FOR COMMENTS — This document is a work in progress.

1. Overview

This guide covers runtime reconfiguration of the `nginx.http` object — the top-level HTTP subsystem accessible from JavaScript. It covers:

- Reading HTTP subsystem state (`servers[]`, `sockets[]`, `variables{}`, hash-size parameters)
- Provisioning new listening sockets (`nginx.createSocket()`)
- Wiring sockets into the HTTP pipeline (`nginx.http.attach()`)
- Managing server-name dispatch (`addServer`, `removeServer`, `rebuildVhostDispatch`, `match`)
- Safe patterns for cross-worker propagation

For location-level operations see [Guide 4 \(Locations\)](#). For server-level operations see [Guide 5 \(Servers\)](#). For cross-worker coordination see [Guide 2 \(All Workers\)](#). For snapshot/restore see [Guide 3 \(Checkpoint/Restore\)](#).

1.1. nginx.http at a glance

```
const h = nginx.http;

// Read-only state
h.servers[]           // NginxServer[]
h.sockets[]           // NginxSocket[] – all listening sockets
h.variables{}         // { name: { index, writable } }
h.serverNamesHashMaxSize // number (from server_names_hash_max_size)
h.serverNamesHashBucketSize // number (from server_names_hash_bucket_size)
h.variablesHashMaxSize // number (from variables_hash_max_size)
h.variablesHashBucketSize // number (from variables_hash_bucket_size)

// Mutations
h.addServer(name[, opts]) // → NginxServer
h.removeServer(name)      // void
h.rebuildVhostDispatch()  // void – activates addServer/removeServer
h.match(uri[, serverName]) // → { server, location } | null

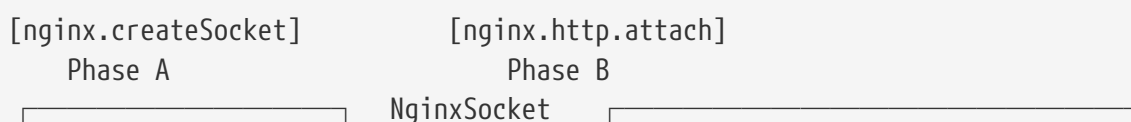
// Socket provisioning
nginx.createSocket('host:port') // → NginxSocket (Phase A)
h.attach(sock)                  // → NginxHttpListener (Phase B)
```

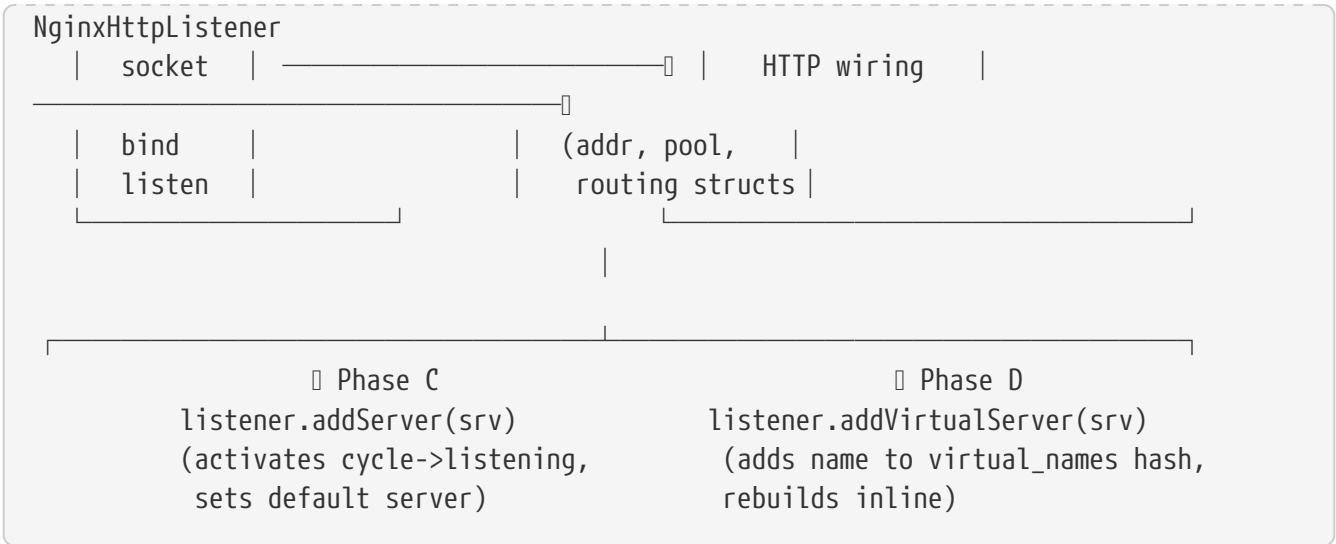
1.2. Lifecycle phases

Socket provisioning spans four phases. Phases A-D are all pre-fork (config phase / `init_conf`). Post-fork socket creation uses Phase F.

Phase A	<code>nginx.createSocket('addr:port')</code>	→ <code>NginxSocket</code>
Phase B	<code>nginx.http.attach(sock)</code>	→ <code>NginxHttpListener</code>
Phase C	<code>listener.addServer(srv)</code>	activates the listener
Phase D	<code>listener.addVirtualServer(srv)</code>	adds a virtual-host entry
Phase F	<code>nginx.createSocket('addr:port')</code>	→ <code>NginxSocket</code> (post-fork, via manager <code>SCM_RIGHTS</code>)

The four-phase model exists because nginx's listening-socket infrastructure, HTTP connection pipeline, and virtual-host hash are built up incrementally. Each `addServer` / `addVirtualServer` call triggers an inline hash rebuild so the new names are reachable immediately.





2. nginx.createSocket(address)

Creates a new TCP listening socket.

2.1. Signature

```
nginx.createSocket(address: string): NginxSocket
```

address — "host:port" string, e.g. "0.0.0.0:9000" or "[::]:443".

2.2. Behaviour

- **Phase A (pre-fork / `init_conf`)** — calls `socket()`, `bind()`, and `listen()` directly in the master process. The socket is ready before any worker forks.
- **Phase F (post-fork, from a request handler)** — sends the bind address to the manager thread via `AF_UNIX + SCM_RIGHTS`. The manager creates the socket in the master's network namespace and delivers the file descriptor back to the requesting worker. Other workers obtain the same fd via `sock.broadcast()`.

2.3. NginxSocket properties and methods

Member	R/W	Description
<code>address</code>	R	Bind address string, e.g. "0.0.0.0".
<code>port</code>	R	Bind port number.
<code>fd</code>	R	Raw file descriptor number.
<code>listener</code>	R	Associated <code>NginxHttpListener</code> , or <code>null</code> if <code>attach()</code> has not been called.
<code>close()</code>	—	Closes the socket file descriptor.

Member	R/W	Description
<code>broadcast()</code>	—	(Phase F) Delivers the socket fd to all other workers via the manager thread. Returns <code>Promise<undefined></code> .

2.4. Example: create a socket in config phase

```
// In js_source script, executed during init_conf

const sock = nginx.createSocket('0.0.0.0:9000');
nginx.log(`created socket fd=${sock.fd} on ${sock.address}:${sock.port}`);
```

3. sock.broadcast()

Distributes a post-fork socket file descriptor to every other worker in the nginx pool so that all workers can accept connections on the new port.

3.1. Signature

```
sock.broadcast(): undefined
```

- Worker-only. Throws `InternalError` if called from config phase.
- Synchronous from the **sender's** perspective — blocks until the manager has delivered the fd and acked.
- Returns `undefined` (not a Promise).

3.2. Why broadcast is needed

In Phase A (pre-fork), `nginx.createSocket()` runs in the master process. The OS socket is created before `fork()`, so every worker inherits the same fd automatically.

In Phase F (post-fork), a worker asks the manager thread to create the socket. The manager creates it in the master's network namespace and returns the fd via `SCM_RIGHTS` to the requesting worker only. Every other worker has its own separate fd table and knows nothing about the new socket until `sock.broadcast()` is called.



```

fork()
|
|----- Worker 0 (fd inherited)
|----- Worker 1 (fd inherited)
|----- Worker 2 (fd inherited)

```

Worker 0 owns fd
Workers 1,2,... know nothing

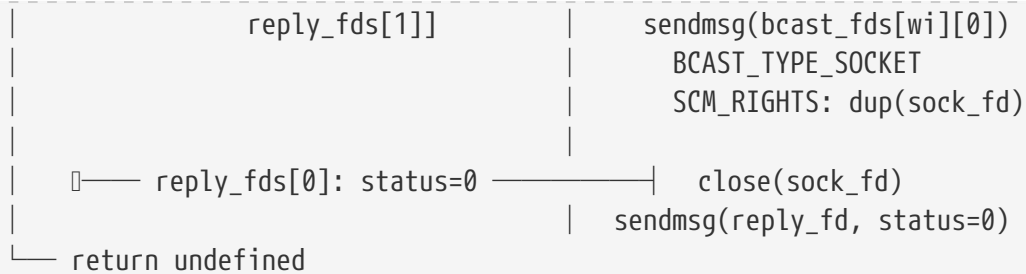
broadcast() fills these

3.3. Full broadcast protocol

`sock.broadcast()` goes through four sub-phases:

- F1 Pre-fork setup (manager start)
`ngx_js_sw_manager_start()` creates one socketpair per worker:
`bcast_fds[wi][0]` – master/manager writes
`bcast_fds[wi][1]` – worker reads
Workers inherit [1] on fork; master keeps [0].
- F2 Worker requests create (`nginx.createSocket` post-fork)
Worker → Manager via `sw_cmd_fds`:
`sendmsg: [0:u32][CMD_CREATE_SOCKET:u32][worker_idx:u32]`
`[addr_len:u32][addr:bytes]`
`SCM_RIGHTS: reply_fd`
Manager: `bind+listen` → sends fd back via `reply_fd` `SCM_RIGHTS`.
Worker now owns the only copy of the new socket fd.
- F3 Worker registers socket locally
`ngx_js_socket_reg[handle] = st;` (global table, COW)
`w->local_socket_reg[i] = st;` (cleanup tracking)
- F4 `sock.broadcast()` – distribute to siblings
Worker → Manager via `sw_cmd_fds`:
`sendmsg: [0:u32][CMD_BROADCAST_SOCKET:u32][worker_idx:u32]`
`[handle:u32][addr_len:u32][addr:bytes]`
`SCM_RIGHTS: [sock_fd, reply_fd]`
Manager iterates workers `wi` → sender:
`sendmsg(bcast_fds[wi][0]):`
`[BCAST_TYPE_SOCKET:u8][handle:u32][addr_len:u32][addr:bytes]`
`SCM_RIGHTS: sock_fd` (dup sent to each worker)
Manager closes its own copy of `sock_fd`.
Manager → Worker: status byte 0 (success) on `reply_fd`.
`broadcast()` returns.

Worker 0 (sender)	Manager thread
<code>sock.broadcast()</code>	
— <code>sendmsg(sw_cmd_fds[1])</code>	— <code>recvmsg(sw_cmd_fds[0])</code>
<code>CMD_BROADCAST_SOCKET</code>	
<code>SCM_RIGHTS: [sock_fd,</code>	<code>for wi in 0..N, wi != 0:</code>



Worker 1 (receiver, async)

bcast_rcv_handler fires when
bcast_fds[1][1] is readable:
recvmsg → rcv_fd
alloc ngx_js_socket_state_t
ngx_js_socket_reg[handle] = st
w->local_socket_reg[i] = st
ngx.onSocket(sock) ← if set

Worker 2 (receiver, async)
└─ (same as Worker 1)

3.4. ngx.onSocket — the receiver callback

When a worker receives a broadcast socket fd via `bcast_rcv_handler`, the C layer:

1. Creates a `ngx_js_socket_state_t` entry for the received fd.
2. Registers it in `ngx_js_socket_reg[handle]` and `local_socket_reg`.
3. Looks up `ngx.onSocket` in the JS global object.
4. If `ngx.onSocket` is a function, calls `ngx.onSocket(sock)` with a freshly-wrapped `NginxSocket` JS object.

This lets receiving workers act on the new socket immediately:

```
// Registered once in js_source (shared by all workers)

ngx.onSocket = function(sock) {
  ngx.log(8, `worker received broadcast socket ${sock.address}:${sock.port}`
    + ` fd=${sock.fd}`);
  // sock is a fully usable NginxSocket in this worker
  // e.g. start an accept loop, track it, close it, etc.
};
```



`ngx.onSocket` is called synchronously within `bcast_rcv_handler`. If it throws, the exception is logged and execution continues. The `NginxSocket` argument is a fresh JS object wrapping the same registry handle used by the sender.

3.5. Timing and sequencing

Side	Behaviour
Sender (Worker 0)	<code>broadcast()</code> blocks (in C) until the manager has finished distributing the fd to all workers and sent its ack byte. The <code>sendmsg</code> to each worker via <code>bcast_fds[wi][0]</code> is non-blocking from the manager's point of view, but the manager iterates all workers before replying.
Receiver (Worker N)	The <code>bcast_recv_handler</code> fires when <code>bcast_fds[wi][1]</code> becomes readable — i.e. on the next epoll iteration after the manager's <code>sendmsg</code> . This is asynchronous from the sender's perspective. <code>broadcast()</code> returning in Worker 0 does NOT guarantee Worker N has already called <code>nginx.onSocket</code> .

If you need all workers to have acted on the socket before proceeding, combine `broadcast()` with `suspendAllWorkers()` / `resumeAllWorkers()`:

```
async function broadcastAndSync(sock) {
  // Suspend all workers so the bcast channel drains before
  // any new connection is accepted on the new port
  await nginx.suspendAllWorkers();
  try {
    sock.broadcast();
    // Manager has sent fd to all workers; their event loops
    // will drain the bcast fd before the first accept event
    // fires, because accept events are suspended.
  } finally {
    await nginx.resumeAllWorkers();
  }
}
```

3.6. The bcast channel is activated lazily

The per-worker bcast fd (`bcast_fds[wi][1]`) is stored in `w->bcast_fd` at `init_process`. The epoll read event for it is **not** registered immediately — it is registered the first time a JS content handler runs via `ngx_js_bcast_ensure_active(w)`. This means:

- A broadcast sent before any request has been handled on Worker N will be buffered in the AF_UNIX socket kernel buffer. It will be delivered as soon as Worker N handles its first request (and activates the event).
- The kernel buffer is large enough for hundreds of queued messages; in practice this is never a concern.

3.7. Errors and constraints

Condition	Result
Called from config phase (master)	Throws <code>InternalError: sock.broadcast: only valid in worker processes</code>
Socket already closed	Throws <code>InternalError: sock.broadcast: socket already closed or invalid</code>
Manager not running	Throws <code>InternalError: sock.broadcast: manager failed to distribute socket to workers</code>
Registry slot collision on receiver	Logs <code>WARN: bcast: handle N already occupied</code> and discards the received fd

3.8. Example: minimal post-fork port provisioning

```
// Registered in js_source – called in every worker via nginx.broadcast()

nginx.onSocket = function(sock) {
  nginx.log(7, `new socket: port ${sock.port}`);
  // The socket is bound and listening; start using it
};

// A request handler that provisions a new port on demand

async function provisionHandler(req) {
  const port = req.variables.arg_port || '9090';

  // Phase F2/F3: manager creates socket, returns fd to this worker
  const sock = nginx.createSocket(`0.0.0.0:${port}`);

  // Phase F4: manager distributes fd to all other workers;
  //           each worker calls nginx.onSocket(sock) asynchronously
  sock.broadcast();

  req.respond(200, {}, `port ${sock.port} provisioned on all workers\n`);
}
```

3.9. Example: broadcast with confirmation via SharedWorker

Because receivers are notified asynchronously, a SharedWorker can act as a rendezvous point to know when all workers have received the socket:

```
// -- shared-worker.js --
```

```

let pending = null;

self.onmessage = function(e) {
  const msg = e.data;
  if (msg.type === 'socket-ready') {
    // Worker reports it received a broadcast socket
    if (pending && pending.port === msg.port) {
      pending.count--;
      if (pending.count === 0) {
        // All workers have the socket
        self.postMessage({ type: 'all-ready', port: msg.port });
        pending = null;
      }
    }
  } else if (msg.type === 'expect') {
    pending = { port: msg.port, count: msg.workerCount };
  }
};

// -- main js_source --

const sw = new SharedWorker('/shared-worker.js');

nginx.onSocket = function(sock) {
  // Tell SharedWorker this worker is ready
  sw.postMessage({ type: 'socket-ready', port: sock.port });
};

async function provisionWithConfirm(req) {
  const port = parseInt(req.variables.arg_port || '9090');

  // Tell SW how many workers to wait for
  sw.postMessage({ type: 'expect', port,
    workerCount: nginx.cycle.workers });

  const sock = nginx.createSocket(`0.0.0.0:${port}`);
  sock.broadcast();

  // Wait for SW to confirm all workers are ready
  await new Promise(resolve => {
    sw.onmessage = function(e) {
      if (e.data.type === 'all-ready' && e.data.port === port) {
        resolve();
      }
    };
  });

  req.respond(200, {}, `port ${port} ready on all workers\n`);
}

```

3.10. Example: broadcast + suspend for atomic rollout

Combine `broadcast()` with `withSuspendedAcceptance()` to guarantee that every worker has registered the new socket **before** it starts accepting connections on any port:

```
async function atomicPortRollout(req) {
  const port = req.variables.arg_port || '7070';

  let sock;

  await nginx.withSuspendedAcceptance(async () => {
    // All workers paused – no new connections accepted anywhere

    // Phase F: create socket in this worker
    sock = nginx.createSocket(`0.0.0.0:${port}`);

    // Distribute to all workers while they are suspended.
    // Their bcast event handlers will fire before the first accept
    // because accept events are still disabled.
    sock.broadcast();

    // All workers will have the socket registered in their
    // local_socket_reg[] by the time resume fires.
  });
  // All workers now resumed, new socket is available everywhere

  req.respond(200, {}, `port ${port} rolled out\n`);
}
```

Worker 0

```
├─ await nginx.withSuspendedAcceptance(async () => {
│   │   suspendAllWorkers() → manager → all workers ack
│   │   └─ nginx.createSocket('0.0.0.0:7070') [Phase F]
│   │       └─ manager creates socket, returns fd
│   └─ sock.broadcast() [Phase F4]
│       └─ manager sends fd to bcast_fds[1][0],
│           bcast_fds[2][0], ...
│           (buffered; workers are suspended)
│
│   resumeAllWorkers() → manager → all workers ack
│   Workers drain bcast fd BEFORE first epoll accept event
│   → nginx.onSocket(sock) called in each worker
└─ req.respond(200, {}, 'done\n')
```

4. nginx.http.attach(sock)

Wires a `NginxSocket` into the HTTP connection pipeline.

4.1. Signature

```
nginx.http.attach(sock: NginxSocket): NginxHttpListener
```

4.2. Behaviour

- Pre-fork only. Calling `attach()` post-fork throws an error.
- Fills in the `sockaddr`, connection pool, and HTTP routing structures for the given socket.
- Returns a `NginxHttpListener` that can be used to register servers (Phase C) and virtual hosts (Phase D).
- Sets `sock.listener` to the returned listener.

4.3. NginxHttpListener properties and methods

Member	R/W	Description
<code>address</code>	R	Bind address string.
<code>socket</code>	R	The <code>NginxSocket</code> this listener was created from.
<code>serverNames</code>	R	Array of all server-name strings registered on this listener.
<code>serverByName(name)</code>	—	Returns the <code>NginxServer</code> registered under <code>name</code> , or <code>null</code> .
<code>addServer(srv)</code>	—	Phase C — sets <code>srv</code> as the default server for this listener and activates it by pushing to <code>cycle→listening</code> . Must be called exactly once per listener before <code>addVirtualServer</code> .
<code>addVirtualServer(srv)</code>	—	Phase D — adds <code>srv</code> as a named virtual host. Rebuilds the virtual-names hash inline. May be called multiple times.

4.4. Full Phase A-D example

```
// Phase A – create socket
const sock = nginx.createSocket('0.0.0.0:8888');

// Phase B – attach to HTTP pipeline
const listener = nginx.http.attach(sock);

// Build a default server
const defaultSrv = nginx.http.addServer('_');
defaultSrv.addLocation('/').handler = function(req) {
    req.respond(444, {}, ''); // close silently
```

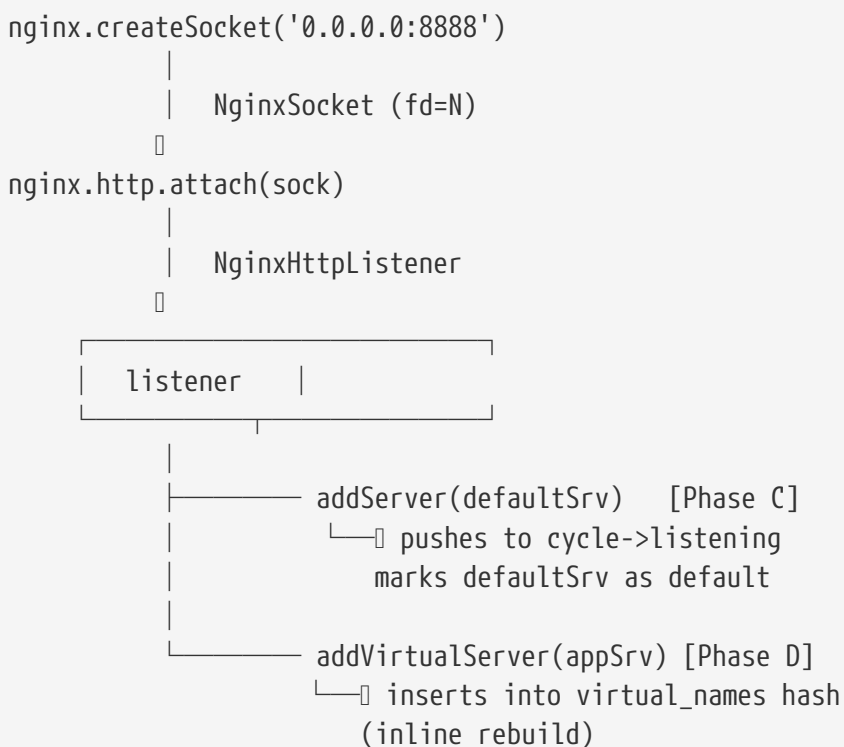
```
};

// Phase C – activate listener with a default server
listener.addServer(defaultSrv);

// Build a named virtual host
const appSrv = nginx.http.addServer('app.example.com');
appSrv.addLocation('/').handler = function(req) {
    req.respond(200, {'Content-Type': 'text/plain'}, 'hello from app\n');
};
nginx.http.rebuildVhostDispatch();

// Phase D – wire named virtual host into this listener
listener.addVirtualServer(appSrv);
```

4.4.1. Flow diagram



4.5. Post-fork socket creation (Phase F)

After workers have forked, `nginx.createSocket()` routes through the manager thread. Because each worker has its own fd table the socket must be explicitly distributed with `sock.broadcast()`.

```
// In a request handler (post-fork)

async function provisionSocket(req) {
    // Ask manager to create + bind + listen
    const sock = await nginx.createSocket('0.0.0.0:9001');
```

```
// Deliver fd to sibling workers
await sock.broadcast();

req.respond(200, {}, `socket ready on port ${sock.port}\n`);
}
```



`attach()` is pre-fork only. Post-fork dynamic sockets must be managed at a lower level (e.g. passed to a `SharedWorker` that owns the accept loop).

5. `nginx.http.servers[]`

Read-only array of all `NginxServer` objects currently known to the HTTP subsystem.

```
for (const srv of nginx.http.servers) {
  nginx.log(`server: names=[${srv.names}] locations=${srv.locations.length}`);
}
```

The array reflects both statically-configured servers and servers added at runtime via `addServer()`. Order matches the order servers were registered.

6. `nginx.http.sockets[]`

Read-only array of all listening socket descriptors with their HTTP configuration.

Each entry exposes:

Property	Type	Description
<code>address</code>	string	Bind address (e.g. <code>"0.0.0.0"</code>).
<code>port</code>	number	Bind port.
<code>ssl</code>	boolean	Whether TLS is active on this listener.
<code>http2</code>	boolean	Whether HTTP/2 is negotiated on this listener.
<code>listen</code>	array	Raw listen-directive parameter objects for this port.

```
for (const s of nginx.http.sockets) {
  nginx.log(`${s.address}:${s.port} ssl=${s.ssl} http2=${s.http2}`);
}
```

7. `nginx.http.variables{}`

Plain JavaScript object mapping every registered HTTP variable name to its metadata.

```
{
  "host":          { index: 0,  writable: false },
  "request_method": { index: 1,  writable: false },
  "my_custom_var": { index: 42, writable: true  },
  ...
}
```

Fields:

Field	Type	Description
<code>index</code>	number	Internal variable index used by nginx's variable engine.
<code>writable</code>	boolean	Whether the variable can be set at request time (e.g. <code>set \$my_var ...</code>).

This map is populated at config parse time and is read-only at runtime. It lets JavaScript code discover which variables are available and whether they can be written.

```
// Check whether a variable exists and is writable
function varInfo(name) {
  const v = nginx.http.variables[name];
  if (!v) return `${name}: not registered`;
  return `${name}: index=${v.index} writable=${v.writable}`;
}
nginx.log(varInfo('host'));
nginx.log(varInfo('my_custom_var'));
```

8. Hash size parameters

These four properties mirror the corresponding `nginx.conf` directives. They are read-only at runtime; changing them requires a config reload.

Property	nginx.conf directive	Purpose
<code>serverNamesHashMaxSize</code>	<code>server_names_hash_max_size</code>	Maximum hash table size for server-name dispatch. Increase if many virtual hosts cause hash-bucket conflicts.
<code>serverNamesHashBucketSize</code>	<code>server_names_hash_bucket_size</code>	Bucket size (bytes, power-of-two). Increase if server names are long.
<code>variablesHashMaxSize</code>	<code>variables_hash_max_size</code>	Maximum hash table size for the variable engine.
<code>variablesHashBucketSize</code>	<code>variables_hash_bucket_size</code>	Bucket size for the variable hash.

```
nginx.log(
  `serverNames hash: max=${nginx.http.serverNamesHashMaxSize}` +
```

```
` bucket=${nginx.http.serverNamesHashBucketSize}`  
);
```

9. nginx.http.addServer(name[, opts])

Adds a new virtual server to the HTTP subsystem.

9.1. Signature

```
nginx.http.addServer(name: string, opts?: { template?: string }): NginxServer
```

name — primary server name. **opts.template** — optional name of an existing server to copy configuration from (inherits **proxy_pass**, SSL, etc.). Defaults to an empty configuration.

The new server starts with an empty location tree. See [Guide 5](#) for full details.

9.2. Two-phase commit

addServer() registers the server internally but **does not** update the server-name routing hash until **rebuildVhostDispatch()** is called. Incoming requests continue to use the old dispatch table until the rebuild.

```
nginx.http.addServer('new.example.com')  
    |  
    | registered internally  
    | (not yet routable)  
    |  
nginx.http.rebuildVhostDispatch()  
    |  
    └─ routing hash rebuilt – new.example.com now reachable
```

```
const srv = nginx.http.addServer('api.example.com');  
srv.addLocation('/v1').handler = function(req) {  
    req.respond(200, {}, 'v1\n');  
};  
nginx.http.rebuildVhostDispatch(); // activate
```

10. nginx.http.removeServer(name)

Removes a virtual server from the HTTP subsystem.

10.1. Signature

```
nginx.http.removeServer(name: string): void
```

Splices the named server from all internal virtual-host entries. Like `addServer`, the routing hash is not updated until `rebuildVhostDispatch()`.

In-flight requests that are already executing inside the removed server's handlers continue to completion. New connections will not be dispatched to the removed server after the next `rebuildVhostDispatch()`.

```
nginx.http.removeServer('api.example.com');  
nginx.http.rebuildVhostDispatch();
```

11. nginx.http.rebuildVhostDispatch()

Rebuilds the server-name routing hash from the current `servers[]` list.

11.1. Signature

```
nginx.http.rebuildVhostDispatch(): void
```

Must be called after any `addServer()` or `removeServer()` to make the changes reachable by incoming requests. Safe to call at any time — the hash rebuild is an $O(N)$ operation and completes in microseconds for typical deployments.

```
nginx.http.addServer('a.example.com');  
nginx.http.addServer('b.example.com');  
nginx.http.rebuildVhostDispatch(); // both become reachable atomically
```

12. nginx.http.match(uri[, serverName])

Simulates the full nginx dispatch logic and returns the server and location that would handle a given request.

12.1. Signature

```
nginx.http.match(uri: string, serverName?: string): { server: NginxServer, location:  
NginxLocation } | null
```

uri — request URI path (e.g. `/api/v1/users`). **serverName** — optional **Host** header value for virtual-host selection. Defaults to the first (default) server.

Returns **null** if no match is found.

```
const m = nginx.http.match('/api/v1/users', 'api.example.com');
if (m) {
  nginx.log(`server=${m.server.names[0]} location=${m.location.path}`);
}
```

Useful for integration testing config changes without sending real requests:

```
function assertRoutes(routes) {
  for (const [uri, host, wantPath] of routes) {
    const m = nginx.http.match(uri, host);
    if (!m || m.location.path !== wantPath) {
      throw new Error(`match(${uri}, ${host}) → ${m?.location.path} ≠
${wantPath}`);
    }
  }
}

assertRoutes([
  ['/api/v1/users', 'api.example.com', '/api/v1/'],
  ['/health', 'api.example.com', '/health'],
  ['/', 'www.example.com', '/'],
]);
```

13. Complete examples

13.1. Example 1: Dynamic HTTP endpoint provisioning

Add a new API server at runtime from a request handler.

```
// In js_source script

(function() {
  const loc = nginx.http.servers[0].locations.find(l => l.path === '/admin');
  if (loc) loc.handler = adminHandler;
})();

async function adminHandler(req) {
  const body = await req.text();
  let cmd;
  try { cmd = JSON.parse(body); } catch(e) {
    req.respond(400, {}, 'bad JSON\n'); return;
  }
```

```

}

if (cmd.action === 'add-server') {
  const srv = nginx.http.addServer(cmd.name);

  for (const [path, msg] of Object.entries(cmd.routes || {})) {
    const capture = msg;
    srv.addLocation(path).handler = function(r) {
      r.respond(200, {'Content-Type': 'text/plain'}, capture + '\n');
    };
  }

  nginx.http.rebuildVhostDispatch();
  req.respond(200, {}, `added server ${cmd.name}\n`);
} else if (cmd.action === 'remove-server') {
  nginx.http.removeServer(cmd.name);
  nginx.http.rebuildVhostDispatch();
  req.respond(200, {}, `removed server ${cmd.name}\n`);
} else {
  req.respond(400, {}, `unknown action ${cmd.action}\n`);
}
}

```

Usage:

```

curl -X POST http://localhost/admin \
  -d '{"action":"add-server","name":"new.example.com",
      "routes":{"":"/":"hello","/api/":"api handler"}}'

curl -H 'Host: new.example.com' http://localhost/
# → hello

```

13.2. Example 2: Inspect all sockets and servers at startup

```

(function auditHttp() {
  nginx.log('=== HTTP Subsystem Audit ===');

  nginx.log(`Listening sockets (${nginx.http.sockets.length}):`);
  for (const s of nginx.http.sockets) {
    nginx.log(`  ${s.address}:${s.port}  ssl=${s.ssl} http2=${s.http2}`);
  }

  nginx.log(`Servers (${nginx.http.servers.length}):`);
  for (const srv of nginx.http.servers) {

```

```

    nginx.log(` [${srv.names.join(', ')}] locations=${srv.locations.length}`);
  }

  nginx.log(`Hash sizes:`);
  nginx.log(`  serverNames max=${nginx.http.serverNamesHashMaxSize}` +
    `  bucket=${nginx.http.serverNamesHashBucketSize}`);
  nginx.log(`  variables  max=${nginx.http.variablesHashMaxSize}` +
    `  bucket=${nginx.http.variablesHashBucketSize}`);

  const vars = nginx.http.variables;
  const writable = Object.entries(vars).filter(([v]) => v.writable).map(([n]) =>
n);
  nginx.log(`Variables: total=${Object.keys(vars).length}
writable=${writable.length}`);
  if (writable.length) nginx.log(`  writable: ${writable.join(', ')}`);
})();

```

13.3. Example 3: Provision a new port at config time (Phases A-D)

```

// Full Phase A-D: new port with two virtual hosts

// Phase A
const sock = nginx.createSocket('0.0.0.0:8888');

// Phase B
const listener = nginx.http.attach(sock);

// Build servers
const defaultSrv = nginx.http.addServer('_');
defaultSrv.addLocation('/= /health').handler = function(req) {
  req.respond(200, {}, 'OK\n');
};
defaultSrv.addLocation('/').handler = function(req) {
  req.respond(404, {}, 'not found\n');
};

const apiSrv = nginx.http.addServer('api.internal');
apiSrv.addLocation('/v1/').handler = function(req) {
  req.respond(200, {'Content-Type': 'application/json'}, '{"v":1}\n');
};

// Phase C – activate listener
listener.addServer(defaultSrv);

// Make servers routable
nginx.http.rebuildVhostDispatch();

```

```
// Phase D – add virtual host
listener.addVirtualServer(apiSrv);

nginx.log('Port 8888 ready: default + api.internal');
```

13.4. Example 4: Variable introspection in a request handler

```
// Expose all nginx variables for a given request (debugging endpoint)

function varDumpHandler(req) {
  const vars = nginx.http.variables;
  const lines = [];
  for (const [name, meta] of Object.entries(vars).sort()) {
    try {
      const val = req.variables[name];
      lines.push(`${name}=${JSON.stringify(val)} (idx=${meta.index})`);
    } catch (_) {
      lines.push(`${name}<error> (idx=${meta.index})`);
    }
  }
  req.respond(200, {'Content-Type': 'text/plain'}, lines.join('\n') + '\n');
}
```

13.5. Example 5: Cross-worker server provisioning via SharedWorker

This example shows how to propagate a new server to all workers using a SharedWorker as the coordination hub. See [Guide 2](#) for the full SharedWorker broadcast pattern.

```
// -- shared-worker.js --

const HANDLER_REGISTRY = {}; // name → function

self.onmessage = function(e) {
  const msg = e.data;
  if (msg.type === 'register-handler') {
    HANDLER_REGISTRY[msg.name] = new Function('req', msg.body);
    self.postMessage({ type: 'ack', name: msg.name });
  }
};

// -- main js_source script --

const sw = new SharedWorker('/shared-worker.js');
```

```
// Broadcast handler registration to all workers
function broadcastHandler(name, fn) {
  sw.postMessage({
    type: 'register-handler',
    name: name,
    body: fn.toString().replace(/^function[^\{]*\{|\}$/g, '')
  });
}

// Called on each worker after receiving broadcast
function applyHandlerLocally(name, body) {
  const fn = new Function('req', body);
  const srv = nginx.http.servers.find(s => s.names.includes(name));
  if (srv) {
    srv.locations.find(l => l.path === '/').handler = fn;
  }
}
```

13.6. Example 6: match() as a config validation tool

```
// Validate routing table after a bulk reconfiguration

function validateRouting(testCases) {
  const failures = [];
  for (const { uri, host, expectedServer, expectedLocation } of testCases) {
    const m = nginx.http.match(uri, host);
    if (!m) {
      failures.push(`${host}${uri}: no match (expected ${expectedLocation})`);
      continue;
    }
    if (expectedServer && !m.server.names.includes(expectedServer)) {
      failures.push(`${host}${uri}: server=${m.server.names[0]} ≠
${expectedServer}`);
    }
    if (expectedLocation && m.location.path !== expectedLocation) {
      failures.push(`${host}${uri}: location=${m.location.path} ≠
${expectedLocation}`);
    }
  }
  if (failures.length) {
    throw new Error('Routing validation failed:\n' + failures.join('\n'));
  }
  nginx.log('Routing validation passed');
}

// After reconfiguration:
validateRouting([
  { uri: '/api/v1/users', host: 'api.example.com',
    expectedServer: 'api.example.com', expectedLocation: '/api/v1/' },
```

```

{ uri: '/health',      host: 'api.example.com',
  expectedLocation: '/health' },
{ uri: '/',            host: 'www.example.com',
  expectedServer: 'www.example.com', expectedLocation: '/' },
]);

```

14. Cross-worker propagation

Changes to `nginx.http` (adding/removing servers, modifying locations) affect only the current worker's view of configuration. To propagate changes to all workers use one of the three strategies from [Guide 2](#).

The diagram below shows the full provisioning pipeline for a post-fork `addServer` broadcast:



JS functions cannot be transferred via `postMessage` — they are not serialisable. Always use a named handler registry: broadcast the **name** string, and have each worker look up the function locally.

14.1. Atomic multi-server reconfiguration across all workers

If you need to add several servers atomically (no request should see a partial state), use `suspendAllWorkers()`:

```

async function atomicMultiServerAdd(specs) {

```

```

    await nginx.suspendAllWorkers();    // all workers pause acceptance
    try {
      for (const { name, routes } of specs) {
        const srv = nginx.http.addServer(name);
        for (const [path, handlerName] of Object.entries(routes)) {
          srv.addLocation(path).handler = HANDLER_REGISTRY[handlerName];
        }
      }
      nginx.http.rebuildVhostDispatch();
    } finally {
      await nginx.resumeAllWorkers();
    }
  }
}

```

See [Guide 2](#) for the `suspendAllWorkers` / `resumeAllWorkers` protocol.

15. API Reference

15.1. nginx (global)

Method / Property	Phase	Description
<code>nginx.createSocket(addr)</code>	A (pre-fork), F (post-fork)	Create, bind, and listen a new TCP socket. Returns <code>NginxSocket</code> .
<code>nginx.http</code>	any	The <code>NginxHttp</code> singleton.

15.2. NginxHttp (nginx.http)

Member	R/W	Description
<code>servers[]</code>	R	Array of all <code>NginxServer</code> objects.
<code>sockets[]</code>	R	Array of listening socket descriptors with HTTP metadata.
<code>variables{}</code>	R	Map of all registered HTTP variables: <code>{ name: { index, writable } }</code> .
<code>serverNamesHashMaxSize</code>	R	<code>server_names_hash_max_size</code> directive value.
<code>serverNamesHashBucketSize</code>	R	<code>server_names_hash_bucket_size</code> directive value.
<code>variablesHashMaxSize</code>	R	<code>variables_hash_max_size</code> directive value.
<code>variablesHashBucketSize</code>	R	<code>variables_hash_bucket_size</code> directive value.
<code>addServer(name[, opts])</code>	W	Add a server. Not routable until <code>rebuildVhostDispatch()</code> .
<code>removeServer(name)</code>	W	Remove a server. Takes effect after <code>rebuildVhostDispatch()</code> .

Member	R/W	Description
<code>rebuildVhostDispatch()</code>	W	Rebuild server-name routing hash.
<code>match(uri[, serverName])</code>	R	Simulate dispatch. Returns { <code>server</code> , <code>location</code> } or <code>null</code> .
<code>attach(sock)</code>	B (pre-fork)	Wire <code>NginxSocket</code> to HTTP pipeline. Returns <code>NginxHttpListener</code> .

15.3. NginxSocket

Member	R/W	Description
<code>address</code>	R	Bind address string.
<code>port</code>	R	Bind port number.
<code>fd</code>	R	File descriptor number.
<code>listener</code>	R	Associated <code>NginxHttpListener</code> or <code>null</code> .
<code>close()</code>	—	Close the socket.
<code>broadcast()</code>	F (post-fork)	Distribute fd to all other workers via the manager thread using <code>NGX_JS_MGR_CMD_BROADCAST_SOCKET</code> + <code>SCM_RIGHTS</code> . Synchronous (blocks until manager acks). Triggers <code>nginx.onSocket(sock)</code> in every receiving worker. Returns <code>undefined</code> .

15.4. nginx.onSocket (callback)

```
nginx.onSocket: function(sock: NginxSocket): void (optional, set by user)
```

If set, called in every worker that receives a socket via `broadcast()`. Invoked synchronously inside `bcast_rcv_handler` after the socket is registered in the worker's local registry.

```
nginx.onSocket = function(sock) {
  nginx.log(7, `received broadcast socket port=${sock.port} fd=${sock.fd}`);
};
```

15.5. NginxHttpListener

Member	R/W	Description
<code>address</code>	R	Bind address string.
<code>socket</code>	R	Associated <code>NginxSocket</code> .
<code>serverNames</code>	R	Array of registered server-name strings.

Member	R/W	Description
<code>serverByName(name)</code>	—	Returns <code>NginxServer</code> or <code>null</code> .
<code>addServer(srv)</code>	C (pre-fork)	Set default server, activate listener.
<code>addVirtualServer(srv)</code>	D (pre-fork)	Add named virtual host, rebuild virtual-names hash.

16. Design notes

16.1. Why four phases?

nginx's listening infrastructure is built incrementally. `socket()` `bind()` + `listen()` (Phase A) must happen before any process forks so all workers inherit the socket. Wiring the socket into the HTTP connection pipeline (Phase B) requires HTTP core data structures that exist only after `http {}` has been parsed. Activating the listener (Phase C) pushes it to `cycle→listening`, which is read by the event loop at worker start. Adding virtual hosts (Phase D) builds the per-listener name hash.

Separating these phases gives precise control over when each piece of infrastructure becomes active.

16.2. Two-phase commit for server names

Server-name routing uses a static hash table. Adding or removing a server requires rebuilding the entire hash. `addServer()` / `removeServer()` defer the hash rebuild so multiple servers can be modified in a single batch before a single `rebuildVhostDispatch()` call makes all changes visible atomically.

16.3. `variables{}` is read-only

The variable map is populated at config parse time by all modules that register variables. It cannot be extended at runtime because the variable index is embedded in compiled location configurations.

16.4. Post-fork socket creation (Phase F)

After workers fork, the master's network namespace is the only one that can `bind()` privileged ports. Phase F uses the manager thread (which runs in the master) as a proxy: the worker sends the address, the manager creates the socket, and returns the fd via `SCM_RIGHTS`. `broadcast()` then distributes it to sibling workers using the same mechanism.

16.5. `broadcast()` is synchronous, `nginx.onSocket` is asynchronous

`sock.broadcast()` blocks the calling worker's C thread until the manager has sent the fd to all peer

workers via `bcast_fds[wi][0]` and replied with a success byte. The manager's `sendmsg` into each worker's bcast pipe is non-blocking (fire-and-forget from the manager's perspective). Receiving workers process the incoming fd the next time their event loop drains `bcast_fds[wi][1]`, which happens on the next epoll iteration after `broadcast()` returns — not before.

If strict ordering is required (all workers must have the socket before any of them can accept on it), use `withSuspendedAcceptance()` to hold all workers' accept loops while the broadcast completes.

16.6. The bcast channel is a pre-fork socketpair

`ngx_js_sw_manager_start()` creates `bcast_fds[wi][0..1]` socketpairs for each worker **before** `fork()`. Workers inherit the read end (`[wi][1]`) on fork; the master (manager thread) keeps the write end (`[wi][0]`). This is a unidirectional, per-worker delivery channel for socket fds, suspend/resume control bytes, and other future broadcast message types. Because it uses `AF_UNIX SOCK_STREAM` the kernel buffers messages until the worker drains them, so messages are never lost even if the worker is busy.